

The `Calendar` class is an abstract class that provides methods for converting between a specific instant in time and a set of calendar fields such as `YEAR`, `MONTH`, `DAY_OF_MONTH`, `HOURL`, and so on, and for manipulating the calendar fields, such as getting the date of the next week.

You are given a date. You just need to write the method, `getDay`, which returns the day on that date. To simplify your task, we have provided a portion of the code in the editor.

Example
`month = 8`
`day = 14`
`year = 2017`

The method should return `MONDAY` as the day on that date.

AUGUST 2017						
SUN	MON	TUE	WED	THU	FRI	SAT
		1	2	3	4	5
6	7	8	9	10	11	12
13	14	15	16	17	18	19
20	21	22	23	24	25	26
27	28	29	30	31		

```
11 public static String findDay(int month, int day, int year) {
12     Calendar calendar = Calendar.getInstance();
13
14     // Month in Calendar starts from 0 (January = 0)
15     calendar.set(year, month - 1, day);
16
17     String[] days = {
18         "SUNDAY",
19         "MONDAY",
20         "TUESDAY",
21         "WEDNESDAY",
22         "THURSDAY",
23         "FRIDAY",
24         "SATURDAY"
25     };
26
27     return days[calendar.get(Calendar.DAY_OF_WEEK) - 1];
28 }
29
30
31
32
33
34
35
36 > public class Solution {
```

Lines: 34 Cols: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!
You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Problem List

Problem List

Submit

Submit

1360. Number of Days Between Two Dates

Solved

Write a program to count the number of days between two dates.

The two dates are given as strings, their format is `YYYY-MM-DD` as shown in the examples.

Example 1:

Input: date1 = "2019-06-29", date2 = "2019-06-30"

Output: 1

Example 2:

Input: date1 = "2020-01-15", date2 = "2019-12-31"

Output: 15

Constraints:

The given dates are valid dates between the years 1971 and 2100.

UNLOCK EXPERIENCES

with a self-driving notebook

HPE

Seen this question in a real interview before?

1/6

Yes No

Accepted 77,082 / 145.2K

Acceptance Rate 53.1%

Topics

19 Online

Code

Java

Auto

1 import java.time.LocalDate;
2 import java.time.temporal.ChronoUnit;
3
4 class Solution {
5 public int daysBetweenDates(String date1, String date2) {
6
7 LocalDate d1 = LocalDate.parse(date1);
8 LocalDate d2 = LocalDate.parse(date2);
9
10 return (int) Math.abs(ChronoUnit.DAYS.between(d1, d2));
11 }
12 }

Saved

In 1, Cal

Testcase

Test Result

Accepted Runtime: 3 ms

Case 1 Case 2

Input

date1 =
"2019-06-29"

date2 =
"2019-06-30"

Output

1

Expected

1

1154. Day of the Year

Easy Topics Companies Hint

Given a string `date` representing a Gregorian calendar date formatted as `YYYY-MM-DD`, return the day number of the year.

Example 1:

Input: `date = "2019-01-09"`
Output: 9
Explanation: Given date is the 9th day of the year in 2019.

Example 2:

Input: `date = "2019-02-10"`
Output: 41

Constraints:

- `date.length == 10`
- `date[4] == date[7] == '-',` and all other `date[i]`'s are digits
- `date` represents a calendar date between Jan 1st, 1900 and Dec 31st, 2019.



Seen this question in a real interview before? 1/5

Yes No

Accepted 101,271 / 202,311 Acceptance Rate 50.1%

517 35 13 Online

Code

Java Auto

```
1 class Solution {
2     public int dayOfYear(String date) {
3         String[] parts = date.split("-");
4
5         int year = Integer.parseInt(parts[0]);
6         int month = Integer.parseInt(parts[1]);
7         int day = Integer.parseInt(parts[2]);
8
9         int[] daysInMonth = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 31, 30, 31};
10
11         // Add days of previous months
12         for (int i = 0; i < month - 1; i++) {
13             day += daysInMonth[i];
14         }
15
16         // Add one day if it's a leap year and after February
17         if (isLeapYear(year) && month > 2) {
18             day++;
19         }
20     }
21 }
```

Saved

Ln 27, Col 2

Testcase Test Result

Accepted Runtime: 0 ms

Case 1 Case 2

Input

date =
"2019-01-09"

Output

9

Expected

9

Contribute a testcase

Problem List

1185. Day of the Week

Description

Editorial

Solutions

Submissions

Run

Ctrl

1185. Day of the Week

EasyTopicsCompasesHint

Given a date, return the corresponding day of the week for that date.
The input is given as three integers representing the `day`, `month` and `year` respectively.
Return the answer as one of the following values ["Sunday", "Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday"].
Note: January 1, 1971 was a Friday.

Example 1:
Input: day = 31, month = 8, year = 2019
Output: "Saturday"

Example 2:
Input: day = 18, month = 7, year = 1999
Output: "Sunday"

Example 3:
Input: day = 15, month = 8, year = 1993
Output: "Sunday"

Constraints:
• The given dates are valid dates between the years 1971 and 2100.

Discover more

Educational Resources

461447 Online

Code

JavaAuto

```
1 import java.time.LocalDate;
2
3 class Solution {
4     public String dayOfTheWeek(int day, int month, int year) {
5
6         LocalDate date = LocalDate.of(year, month, day);
7
8         return date.getDayOfWeek().toString().substring(0, 1)
9             + date.getDayOfWeek().toString().substring(1).toLowerCase();
10     }
11 }
```

Saved

Unit Tests

Testcase

Test Result

AcceptedRuntime: 4 ms

Case 1Case 2Case 3

Input

day =
31

month =
8

year =
2019

Output

"Saturday"

Problem

Submissions

Leaderboard

Discussions

Editorial

HackerRank

PrepareJavaData StructuresJava Priority Queue

Exit Full Screen View

In computer science, a priority queue is an abstract data type which is like a regular queue, but where additionally each element has a "priority" associated with it. In a priority queue, an element with high priority is served before an element with low priority. - Wikipedia

In this problem we will test your knowledge on [Java Priority Queue](#).

There are a number of students in a school who wait to be served. Two types of events, ENTER and SERVED, can take place which are described below.

- ENTER: A student with some priority enters the queue to be served.
- SERVED: The student with the highest priority is served (removed) from the queue.

A unique id is assigned to each student entering the queue. The queue serves the students based on the following criteria (priority criteria):

- The student having the highest Cumulative Grade Point Average (CGPA) is served first.
- Any students having the same CGPA will be served by name in ascending case-sensitive alphabetical order.
- Any students having the same CGPA and name will be served in ascending order of the id.

Create the following two classes:

- The Student class should implement:
 - The constructor `Student(int id, String name, double cgpa)`.
 - The method `int getID()` to return the id of the student.
 - The method `String getName()` to return the name of the student.
 - The method `double getCGPA()` to return the CGPA of the student.
- The Priorities class should implement the method `List<Student> getStudents(List<String> events)` to process all the given events and return all the students yet to be served in the priority order.

Input Format

The first line contains an integer, n , describing the total number of events. Each of the n subsequent lines will be of the following two forms:

- ENTER name CGPA id: The student to be inserted into the priority queue.
- SERVED: The highest priority student in the queue was served.

The locked stub code in the editor reads the input and tests the correctness of the Student and Priorities classes implementation.

Constraints

```
8 private String name;
9 private double cgpa;
10
11 public Student(int id, String name, double cgpa) {
12     this.id = id;
13     this.name = name;
14     this.cgpa = cgpa;
15 }
16
17 public int getID() {
18     return id;
19 }
20
21 public String getName() {
22     return name;
23 }
24
25 public double getCGPA() {
26     return cgpa;
27 }
28 }
29
30 class Priorities {
31
32     public List<Student> getStudents(List<String> events) {
33
34         PriorityQueue<Student> pq = new PriorityQueue<Student>((11, new Comparator<Student>()) {
35             @Override
36             public int compare(Student s1, Student s2) {
```

Line: 34 Col: 94

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Sample Test case 0

Test Case (1/1)

Download

Sometimes it's better to use dynamic size arrays. Java's `ArrayList` can provide you this feature. Try to solve this problem using `ArrayList`.

You are given n lines. In each line there are zero or more integers. You need to answer a few queries where you need to tell the number located in y^{th} position of x^{th} line.

Take your input from `System.in`.

Input Format

The first line has an integer n . In each of the next n lines there will be an integer d denoting number of integers on that line and then there will be d space-separated integers. In the next line there will be an integer q denoting number of queries. Each query will consist of two integers x and y .

Constraints

- $1 \leq n \leq 20000$
- $0 \leq d \leq 50000$
- $1 \leq q \leq 1000$
- $1 \leq x \leq n$

Each number will fit in signed integer.

Total number of integers in n lines will not cross 10^5 .

Output Format

In each line, output the number located in y^{th} position of x^{th} line. If there is no such position, just print "ERROR"

Sample Input

```
5
5 41 77 74 22 44
1 12
4 37 34 36 52
0
3 20 22 33
5
1 3
3 4
3 1
4 3
5 5
```

```
public class Solution {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        ArrayList<ArrayList<Integer>> list = new ArrayList<>();

        // Store the lines of integers
        for (int i = 0; i < n; i++) {
            int d = sc.nextInt();

            ArrayList<Integer> row = new ArrayList<>();

            for (int j = 0; j < d; j++) {
                row.add(sc.nextInt());
            }

            list.add(row);
        }

        int q = sc.nextInt();

        // Answer queries
        for (int i = 0; i < q; i++) {
            int x = sc.nextInt();
            int y = sc.nextInt();

            if (x > 0 && x <= list.size() && y > 0 && y <= list.get(x - 1).size()) {
                System.out.println(list.get(x - 1).get(y - 1));
            } else {
                System.out.println("ERROR");
            }
        }
    }
}
```

Line: 46 Col: 1

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

Comparators are used to compare two objects. In this challenge, you'll create a comparator and use it to sort an array. The `Player` class is provided for you in your editor. It has **2** fields: a `name` `String` and a `score` `Integer`.

Given an array of `n` `Player` objects, write a comparator that sorts them in order of decreasing score; if **2** or more players have the same score, sort those players alphabetically by name. To do this, you must create a `Checker` class that implements the `Comparator` interface, then write an `int compare(Player a, Player b)` method implementing the `Comparator.compare(T o1, T o2)` method.

Input Format

Input from `stdin` is handled by the locked stub code in the `Solution` class.

The first line contains an integer, `n`, denoting the number of players.

Each of the `n` subsequent lines contains a player's `name` and `score`, respectively.

Constraints

- $0 \leq \text{score} \leq 1000$
- 2** players can have the same name.
- Player names consist of lowercase English letters.

Output Format

You are not responsible for printing any output to `stdout`. The locked stub code in `Solution` will create a `Checker` object, use it to sort the `Player` array, and print each sorted element.

Sample Input

```
5
amy 100
david 100
heraldo 50
aakansha 75
alekxa 150
```

Sample Output

```
alekxa 150
amy 100
```

```
18 // Sort by decreasing score
19 if (a.score != b.score) {
20     return b.score - a.score;
21 }
22
23 // If scores are same, sort alphabetically by name
24 return a.name.compareTo(b.name);
25 }
26
27
28 class Player {
29     String name;
30     int score;
31
32     Player(String name, int score) {
33         this.name = name;
34         this.score = score;
35     }
36 }
37
38 class Solution {
39
40     public static void main(String[] args) {
41         Scanner scan = new Scanner(System.in);
42         int n = scan.nextInt();
43
44         Player[] player = new Player[n];
45         Checker checker = new Checker();
```

Lines: 16 Cols: 2

Upload Code as File

☐ Test against custom input

Run Code

Submit Code

Congratulations!

You have passed the sample test cases. Click the submit button to run your code against all the test cases.

☒ Sample Test case 0

View Test Case

Feedback

Problem List

Run Ctrl

Submit

Auto

0

0

0

0

0

Premium

Description

Editorial

Solutions

Submissions

179. Largest Number

Medium Topics Companies

Given a list of non-negative integers `nums`, arrange them such that they form the largest number and return it.

Since the result may be very large, so you need to return a string instead of an integer.

Example 1:

Input: `nums = [10,2]`
Output: `"210"`

Example 2:

Input: `nums = [3,30,34,5,9]`
Output: `"9534330"`

Constraints:

- `1 <= nums.length <= 100`
- `0 <= nums[i] <= 109`

GoDaddy Get email that looks as professional as your business is.

Seen this question in a real interview before? 1/6

9.5K 227 26 Online

Code

Java

Auto

```
1 import java.util.*;
2
3 class Solution {
4     public String largestNumber(int[] nums) {
5         String[] arr = new String[nums.length];
6
7         // Convert integers to strings
8         for (int i = 0; i < nums.length; i++) {
9             arr[i] = String.valueOf(nums[i]);
10        }
11
12        // Sort using custom comparator
13        Arrays.sort(arr, (a, b) -> (b + a).compareTo(a + b));
14
15        // If the largest number is "0", return "0"
16        if (arr[0].equals("0")) {
17            return "0";
18        }
19    }
20 }
```

Saved Ln 28: Col 2

Testcase

Test Result

Accepted Runtime: 1 ms

Case 1 Case 2

Input

nums = [10,2]

Output

210

Problem List

2418. Sort the People

Easy

Topics

Companies

Hint

You are given an array of strings `names`, and an array `heights` that consists of **distinct** positive integers. Both arrays are of length `n`.

For each index `i`, `names[i]` and `heights[i]` denote the name and height of the `i`th person.

Return `names` sorted in **descending order by the people's heights**.

Example 1:

Input: `names = ["Mary", "John", "Emma"], heights = [180, 165, 170]`
Output: `["Mary", "Emma", "John"]`
Explanation: Mary is the tallest, followed by Emma and John.

Example 2:

Input: `names = ["Alice", "Bob", "Bob"], heights = [155, 185, 150]`
Output: `["Bob", "Alice", "Bob"]`
Explanation: The first Bob is the tallest, followed by Alice and the second Bob.

Constraints:

- `n == names.length == heights.length`
- `1 <= n <= 105`
- `1 <= names[i].length <= 20`
- `1 <= heights[i] <= 105`

1.9K 180 6 Online

Code

Java

```
1 import java.util.*;
2
3 class Solution {
4     public String[] sortPeople(String[] names, int[] heights) {
5         int n = names.length;
6
7         // Store (height, name) pairs
8         Integer[] index = new Integer[n];
9         for (int i = 0; i < n; i++) {
10             index[i] = i;
11         }
12
13         // Sort indices based on heights in descending order
14         Arrays.sort(index, (a, b) -> heights[b] - heights[a]);
15
16         // Build the answer
17         String[] result = new String[n];
18         for (int i = 0; i < n; i++) {
19             result[i] = names[index[i]];
20         }
21     }
22 }
```

Saved Ln 24, Col 2

Testcase

Test Result

Accepted Runtime: 1 ms

Case 1 Case 2

Input

names =

["Mary", "John", "Emma"]

heights =

[180, 165, 170]